THE NEUROFORGE

Automated Neural Architecture Discovery & Optimization System

Semester Project Dossier — Statistical Machine Learning

> **Core idea:** Normally, humans design neural-network structures through trial and error. NEUROFORGE automates that design loop by generating candidate architectures, training and evaluating them, and searching for designs that balance predictive performance with computational efficiency.

Primary approach: Supervised Learning + Neural Architecture Search + Evolutionary Optimization + Resource-Aware Evaluation

Initial benchmark: CIFAR-10 **Implementation:** Python + PyTorch

1. Executive Summary

NEUROFORGE is a software-only machine learning system that automatically searches over a defined space of neural-network architectures. It creates candidate designs, trains them under a controlled budget, measures performance and efficiency, and uses an optimization strategy to decide which designs should be explored next.

The project is scoped around small CNNs so the complete pipeline can be implemented on a personal computer within approximately 3.5–4 months. The emphasis is on automation, fair experimentation, architecture search and the accuracy-versus-efficiency trade-off.

> **Research question:** Can automated search discover neural-network architectures that achieve competitive accuracy while using fewer computational resources than manually designed architectures under a fixed computational budget?

2. Problem & Project Scope

Neural-network design involves decisions about layer count, filters/neurons, kernel sizes, activations, normalization, pooling, dropout and skip connections. NEUROFORGE treats these choices as an optimization problem inside a controlled search space.

Important distinction: A model recommender selects an existing model; model configuration tunes an existing architecture family; NAS generates architectures from a defined search space and searches them. NEUROFORGE targets the third concept.

3. System Architecture

> Dataset → preprocessing → train/validation/test → architecture search space → candidate generation → candidate training → evaluation → accuracy + efficiency → fitness/objective → optimization → new candidates → repeat → final architecture(s).

4. Dataset Strategy

CIFAR-10 is the initial benchmark: a public image-classification dataset with 60,000 32×32 RGB images across 10 classes. It is manageable for rapid CNN experiments. A second public dataset can be added later to test generalization.

5. Search Space

Component	Controlled choices
Layers	1–8
Filters	16 / 32 / 64 / 128
Kernel	3×3 / 5×5
Activation	ReLU / GELU / SiLU
Pooling	Yes / No
Dropout	0–0.5
Skip connection	Yes / No
Normalization	BatchNorm / None

6. Evolutionary Search

Architectures can be treated as a population. Each generation is evaluated; promising candidates are selected; mutation and crossover create new candidates; the process repeats.

• **Population:** current collection of candidate architectures.

• **Fitness:** numerical measure used to compare candidates.

• **Selection:** choosing promising candidates as parents.

• **Mutation:** making a small architecture change.

• **Crossover:** combining parts of two parent architectures.

• **Generation:** one search cycle from parent population to evaluated offspring.

7. Resource-Aware Objective

Accuracy should not be optimized alone. A simple V1 score can be expressed as **Score = Accuracy** – λ■**(Model Size)** – λ■**(Latency)**. Later, explicit multi-objective/Pareto analysis can show trade-offs without collapsing everything into one score.

8. NEUROFORGE V1

• Load and preprocess CIFAR-10.

• Train a manually designed baseline CNN.

• Define a limited architecture search space.

• Automatically generate candidate CNNs.

• Train candidates using a controlled proxy budget.

• Record accuracy, loss, parameters, model size and timing.

• Display strong candidates and compare them with manual baselines.

V1 intentionally avoids industrial-scale NAS, huge search spaces and expensive full-length training of thousands of networks.

9. Experimental Design

• **Baselines:** manually designed CNNs versus NEUROFORGE-generated architectures.

• **Metrics:** accuracy, precision, recall, F1, confusion matrix, validation loss, parameters, model size, FLOPs, inference latency and search time.

• **Convergence:** best validation performance across generations/iterations.

• **Ablation:** remove or alter mutation, crossover, resource penalties or skip connections.

• **Generalization:** optionally test finalists on another public dataset.

• **Fair evaluation:** fixed preprocessing, splits, budgets and protocols; held-out test set for final reporting.

10. Four-Month Roadmap

Weeks 1–3: foundation and baseline → **4–6:** CNN study → **7–9:** NAS V1 → **10–12:** evolutionary search → **13–14:** efficiency analysis → **15–16:** ablation, generalization, report and demo.

11. Technical Stack

Python, PyTorch, Apple Metal/MPS where supported, NumPy, Pandas, Matplotlib, scikit-learn, optional MLflow/W&B;, Git and GitHub.

12. Risks & Mitigation

• Too many candidates → use small populations and a controlled search space.

• Search too expensive → proxy training during search and longer retraining only for finalists.

• Unfair comparisons → fixed training budgets and evaluation protocols.

• Validation overfitting → reserve the test set for final evaluation.

• Advanced features consume time → treat RL NAS, compression and attention as optional extensions.

13. Portfolio Framing

Built a resource-aware neural architecture search pipeline that automatically generated and evaluated CNN architectures instead of relying on a single manually designed network.

14. Future Extensions

- Pareto multi-objective optimization
- Hyperparameter optimization
- Knowledge distillation
- Quantization/model compression
- Lightweight attention
- RL-based architecture generation
- Cross-dataset generalization
- Budget-aware search

15. ML Buzzwords & Definitions — Viva Cheat Sheet

Short, professor-friendly definitions of the terminology used throughout NEUROFORGE.

A. Dataset & Training Basics

Term	Definition
Dataset	A collection of examples used for training, validation or testing.
Sample / Example	One individual data point, such as one CIFAR-10 image.
Feature	An input attribute or learned representation used to make a prediction.
Label	The target answer associated with a training example.
Training Set	Data used to learn model parameters.
Validation Set	Data used during development to compare candidates and tune choices.
Test Set	Held-out data used for final evaluation.
Epoch	One complete pass through the training dataset.
Batch	A smaller group of examples processed together.
Batch Size	Number of examples in one batch.
Iteration	One optimizer update step, usually based on one batch.
Learning Rate	Controls the size of parameter updates.
Optimizer	Algorithm that updates parameters to reduce loss, e.g. SGD or Adam.
Loss Function	Mathematical measure of prediction error.
Gradient	Rate and direction of change of loss with respect to parameters.
Gradient Descent	Optimization method that updates parameters to reduce loss.
Backpropagation	Method for computing gradients through network layers.
Parameter	A value learned during training, such as a weight or bias.
Hyperparameter	A setting chosen by the engineer/search process rather than learned directly.
Random Seed	Fixed starting value used to make stochastic experiments more reproducible.

B. Neural Network & CNN

Term	Definition
Neural Network	A parameterized function composed of layers of computational units.
Neuron	Basic computational unit that applies a weighted transformation and activation.
Layer	A stage of computation in a neural network.
Forward Pass	Sending an input through the network to produce an output.
Activation Function	Nonlinear function such as ReLU, GELU or SiLU.
MLP	Multi-Layer Perceptron, mainly built from fully connected layers.
CNN	Convolutional Neural Network, especially useful for image-like grid data.
Convolution	Operation applying learnable filters over local regions of an input.
Kernel / Filter	Learnable weights used by a convolution to detect patterns.
Feature Map	Output produced by applying convolutional filters.
Stride	How many positions a convolution moves at each step.
Padding	Extra border values added around an input.
Pooling	Downsampling operation such as max pooling.
Batch Normalization	Layer-wise normalization technique often used to stabilize training.
Dropout	Regularization that randomly disables some activations during training.
Weight Decay	Regularization that discourages excessively large parameter values.
Data Augmentation	Creating varied training examples using transformations.

Term	Definition
Logits	Raw class scores before conversion to probabilities.
Softmax	Function converting class scores into a probability distribution.
Receptive Field	Input region that can influence a particular neuron.

C. NAS & Evolutionary Optimization

Term	Definition
NAS	Neural Architecture Search: automated search for neural-network architectures.
Architecture	Structural design of a neural network: layers, operations and connections.
Search Space	Set of architectures that the NAS system is allowed to generate.
Search Strategy	Method deciding which architectures to evaluate next.
Candidate Architecture	One specific network design generated from the search space.
Fitness Function	Numerical objective used to compare candidates.
Evolutionary Algorithm	Optimization approach using populations, selection and variation.
Population	Collection of candidate solutions in a generation.
Generation	One cycle of evaluation and reproduction.
Selection	Choosing promising candidates as parents.
Mutation	Small random change to a candidate architecture.
Crossover	Combining parts of two parent architectures.
Exploration	Searching new or less-tested regions of the search space.
Exploitation	Focusing on regions already producing promising results.
Proxy Training	Short, cheaper training used to estimate candidate quality.
Search Convergence	When further search produces little improvement.
Hyperparameter Optimization	Automated search for training settings such as learning rate.
Multi-Objective Optimization	Optimizing several objectives simultaneously.
Pareto Dominance	A candidate is no worse on all objectives and better on at least one.
Pareto Frontier	Set of non-dominated trade-off solutions.
Computational Budget	Predefined limit on resources available for search/training.

D. Evaluation & Efficiency

Term	Definition
Baseline	Reference method used to judge a new approach.
Accuracy	Fraction of predictions that are correct.
Precision	Among predicted positives for a class, fraction that are correct.
Recall	Among actual positives for a class, fraction correctly identified.
F1 Score	Harmonic mean of precision and recall.
Confusion Matrix	Table showing predicted versus actual classes.
Overfitting	Strong training performance but poorer unseen-data performance.
Underfitting	Model is too limited to capture important patterns.
Generalization	Performance on unseen data.
Ablation Study	Experiment removing/changing one component to measure its contribution.
Parameter Count	Number of learnable parameters.
FLOPs	Approximate count of floating-point operations required.
Model Size	Storage required by the saved model.

Term	Definition
Inference	Using a trained model to make predictions.
Inference Latency	Time required to produce a prediction.
Throughput	Amount of data/predictions processed per unit time.
Resource-Aware	Explicitly considering resources such as latency, memory, size or FLOPs.
Reproducibility	Ability to repeat an experiment and obtain comparable results.

16. One-Line Professor Answers

What is NEUROFORGE? An automated NAS system that generates and evaluates CNN designs and uses optimization to find promising architectures.

Why NAS? Neural architecture design contains many structural choices and can be formulated as an optimization problem.

Why evolutionary optimization? It provides a practical way to explore a discrete architecture space using selection, mutation and crossover.

Why CIFAR-10? It is a standard, manageable benchmark that allows rapid CNN experimentation within a semester compute budget.

What is the novelty? The project automates the generation-and-evaluation loop rather than evaluating only one manually selected architecture.

What is the objective? Find strong architectures while explicitly considering efficiency instead of optimizing accuracy alone.

How will you know it works? Compare discovered architectures with manual baselines under controlled budgets, then analyze accuracy, efficiency and search behavior.

Main limitation? The search space is deliberately constrained; this is a research prototype, not a claim of globally optimal architecture discovery.

17. Deliverables

- Working NEUROFORGE codebase
- Dataset and preprocessing pipeline
- Manual CNN baselines
- Architecture encoder/generator
- Evolutionary search algorithm
- Experiment logs
- Best discovered architecture(s)
- Accuracy-versus-efficiency and convergence plots
- Ablation results where feasible
- Final report, presentation and demonstration

18. Final Project Statement

> NEUROFORGE reframes neural-network architecture design as an automated optimization problem. It demonstrates a reproducible system that can generate, train, compare and evolve neural-network designs under controlled resource constraints.